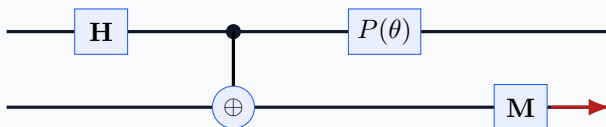


QPU Static Circuit Compiler

Complete Protocol, Gate, and Truth-Table Reference



Bailey Forbes
September 13, 2026

This guide is derived from the current TypeScript AST parser, gate registry, compiler, simulator, and truth-table implementation. It distinguishes syntax that produces executable circuit gates from compatibility syntax that is only recorded in the compiler log.

Contents

1	Scope and execution model	2
1.1	Three levels of support	2
1.2	Current AST command inventory	2
2	Lexical and file syntax	2
2.1	Logical lines, whitespace, and case	2
2.2	Comments	3
2.3	Protocol header	3
2.4	Token references, cycles, and constants	3
2.5	Flags and argument boundaries	3
2.6	Gate arity	4
3	Quantum state mathematics	4
3.1	Measurement	4
3.2	Truth tables versus amplitudes	4
4	Single-qubit primitive gates	4
4.1	Pauli X (X)	5
4.2	Pauli Y (Y)	5
4.3	Pauli Z (Z)	5
4.4	Hadamard (H)	6
4.5	S phase (S)	6
4.6	T phase (T)	6
4.7	Arbitrary phase (PHASE)	7
4.8	Backward-prefix syntax	7
5	Controlled and two-qubit primitive gates	7
5.1	Controlled NOT (CNOT)	7
5.2	Doubly controlled NOT / Toffoli (CCNOT)	8
5.3	Controlled Z (CZ)	8
5.4	Controlled Y (CY)	8
5.5	Swap (SWAP)	9
6	Derived logical gates	9
6.1	Logical NOT (NOT)	9
6.2	Reversible AND (AND)	10
6.3	Reversible NAND (NAND)	10
6.4	Reversible OR (OR)	10
6.5	Reversible XOR (XOR)	11
7	Measurement gate	11
7.1	Measurement (MEASURE)	11

8	State, token, and cycle commands	11
8.1	SET — initialize or alias (lowered/executed)	11
8.2	CREATETOKEN — reserve wires (lowered/executed)	12
8.3	DELETETOKEN and FREE — lifetime hints (parsed only)	12
8.4	INCREASECYCLE — cycle boundary (lowered/executed)	12
8.5	JOIN and SPLIT (parsed only)	12
9	Process composition and return values	12
9.1	MAIN-PROCESS (lowered/executed)	12
9.2	DECLARECHILD (parsed only)	12
9.3	RUNCHILD and CALL (lowered/executed)	13
9.4	RETURNVALS (lowered/executed)	13
9.5	ACCEPTVALS (lowered/executed)	13
9.6	Nested full-adder example	13
10	Accepted compatibility commands	14
11	Truth-table companion syntax	14
12	Complete worked protocol	15
13	Compiler output and lowering details	15
14	Error conditions and practical rules	16
15	Quick-reference grammar	16
16	Operation checklist	17

1 Scope and execution model

The compiler turns a plain-text `.qpucir` protocol into one flat sequence of circuit gates:

protocol source $\rightarrow$ logical lines $\rightarrow$ AST commands $\rightarrow$ scoped tokens $\rightarrow$ gates $\rightarrow$ state-vector simulation.

Child processes are expanded at compile time. The UI and simulator therefore see the same ordered gate list even when the source contains nested processes.

1.1 Three levels of support

Status	Meaning
lowered/executed	The AST accepts the command and the compiler emits behavior that the simulator executes.
parsed only	The AST accepts the command and preserves it in parsed output or the compiler log, but no visual/simulator operation is emitted yet.
compiler-internal	The compiler may emit the operation itself, but protocol source cannot name it directly. <code>RESET</code> is the important example.

1.2 Current AST command inventory

Command	Status	Purpose
X, Y, Z, H, S, T, PHASE	lowered/executed	Single-qubit matrix operations.
CNOT, CCNOT, CZ, CY, SWAP	lowered/executed	Controlled and two-wire primitive operations.
NOT, AND, NAND, OR, XOR	lowered/executed	Derived logical target mutations.
MEASURE	lowered/executed	Measure one named wire or all known wires.
SET, CREATETOKEN	lowered/executed	Initialize/alias tokens and reserve wires.
RUNCHILD, CALL	lowered/executed	Expand a catalog/library child process.
RETURNVALS, ACCEPTVALS	lowered/executed	Bind child and process outputs.
MAIN-PROCESS, INCREASECYCLE	lowered/executed	Mark a process and flush cycle workspace.
FREE, DELETETOKEN, DECLARECHILD	parsed only	Lifetime/declaration hints; currently logged.
JOIN, SPLIT	parsed only	Register-memory compatibility syntax.
COMPILEPROCESS, MASTerval	parsed only	Process-control compatibility syntax.
SAVE_STATE, LOAD_STATE	parsed only	Checkpoint compatibility syntax.

Not source syntax: `RESET` exists in the gate registry, but it is not in the parser's supported-operation list. Write `SET wire 0p`; the compiler schedules internal `RESET` gates for zeroed workspace.

2 Lexical and file syntax

2.1 Logical lines, whitespace, and case

Commands and flags are case-insensitive and are separated by whitespace. Token names retain their spelling. CRLF and LF line endings are accepted. A physical line ending *exactly* in a backslash joins to the next line:

```
PARAMS: A:state B:state \
        Cin:state
```

```
CCNOT -I A B \
      -O Carry
```

The continuation backslash must be the final character; spaces after it prevent continuation. Blank logical lines are discarded.

2.2 Comments

```
# Whole-line or trailing hash comment
H -I Q:0 -O Q:0 # prepare |+>

/* A block comment may span logical lines. */
PHASE=pi/2 -I Q:0 -O Q:0
```

Continuation joining happens before comments are removed. Hash comments run to the end of a logical line. C-style block comments use `/*` and `*/`.

2.3 Protocol header

```
PARAMS: A:state B:state Theta:float Count:int
```

```
MAIN-PROCESS Example
```

`PARAMS:` is recognized only when it is the first non-comment logical line. Each whitespace-separated entry containing a colon becomes `name:type`. The parser does not restrict the type string:

- `state` entries become user-facing circuit inputs and truth-table input columns;
- `int` and `float` entries are retained but excluded from user-facing qubit controls;
- other type names are retained as written.

The first `MAIN-PROCESS name` supplies the process name. If none exists, the AST uses `InlineProcess`.

2.4 Token references, cycles, and constants

Form	Meaning
Q	Bare symbolic token.
\$Q	Parameter/alias spelling; the leading dollar is removed during address resolution.
Q:3	Cycle/bit suffix; everything from the first colon is removed for the current visual wire mapping. Thus <code>Q:0</code> and <code>Q:7</code> map to <code>Q</code> .
0p	Computational basis state $ 0\rangle = [1, 0]^T$.
1p	Computational basis state $ 1\rangle = [0, 1]^T$.
sp	Equal superposition $ +\rangle = (0\rangle + 1\rangle)/\sqrt{2}$.
0p_dimN, 1p_dimN, sp_dimN	Also recognized as constants for an integer <code>N</code> ; the current compiler uses the same one-qubit initialization behavior.

Constants are case-insensitive and shared by canonical constant name. Numeric workspace names passed to a child are scoped to the parent workspace; other child-local names are scoped per invocation.

2.5 Flags and argument boundaries

Gate rows use `-I` and `-O`:

```
GATE -I input1 input2 -O output
```

Each list ends at the next token beginning with a hyphen. Flags are case-insensitive. Positional `args` still contain every token after the opcode, including flags. The AST also records `-$R` as `noParameterSubstitution`; the current

compiler does not act on that flag.

```
PHASE=pi/4 -I Q -O Q -$R
```

2.6 Gate arity

Gate family	Required -I count	Required -O count
X, Y, Z, H, S, T, PHASE, NOT	1	1
CNOT, CZ, CY	1	1
CCNOT, AND, NAND, OR, XOR	2	1
SWAP	2	2
MEASURE	0 or 1	0

All counts are exact except **MEASURE**. For ordinary single-qubit gates, name the same wire in **-I** and **-O**. The output is the mutated target; for controlled gates, **-I** contains controls.

3 Quantum state mathematics

One qubit is a normalized complex vector:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

A gate U acts linearly: $|\psi'\rangle = U|\psi\rangle$. A valid closed-system gate is unitary, $U^\dagger U = I$, so normalization is preserved. For n qubits, basis states are tensor products and the state has 2^n amplitudes:

$$|\Psi\rangle = \sum_{x=0}^{2^n-1} a_x |x\rangle, \quad \sum_x |a_x|^2 = 1, \quad |AB\rangle = |A\rangle \otimes |B\rangle.$$

The simulator stores this state vector directly. A controlled operation applies its target matrix only to basis components whose controls satisfy its predicate.

3.1 Measurement

For a measured qubit q ,

$$P(q=1) = \sum_{x: x_q=1} |a_x|^2, \quad P(q=0) = 1 - P(q=1).$$

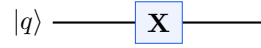
After sampling $b \in \{0,1\}$, incompatible amplitudes are set to zero and the remaining branch is renormalized by $1/\sqrt{P(q=b)}$. Measurement is therefore not a unitary matrix operation.

3.2 Truth tables versus amplitudes

A classical truth table describes deterministic basis-state behavior. It cannot show relative phase or interference. Consequently, the tables below compare basis mappings, while matrices and state equations describe the full quantum action. In companion `.qpuio` tables, cells are `0p`, `1p`, or `sp`; an expected output `sp` is treated as “don’t care” during measured-output comparison.

4 Single-qubit primitive gates

4.1 Pauli X (X)

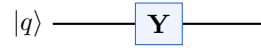


$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad X(\alpha|0\rangle + \beta|1\rangle) = \beta|0\rangle + \alpha|1\rangle.$$

```
X -I Q:0 -0 Q:0
```

Input	Classical comparison	Output
0	NOT 0	1
1	NOT 1	0

4.2 Pauli Y (Y)



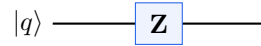
$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle.$$

It flips the basis value like X but adds phases that become observable through later interference.

```
Y -I Q -0 Q
```

Input	Measured bit	Quantum output
0	1	$i 1\rangle$
1	0	$-i 0\rangle$

4.3 Pauli Z (Z)



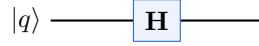
$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle.$$

Z does not change a computational-basis measurement, but it changes relative phase. For example, $Z|+\rangle = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$.

```
Z -I Q -0 Q
```

Input	Classical bit	Quantum output
0	0	$ 0\rangle$
1	1	$- 1\rangle$

4.4 Hadamard (H)



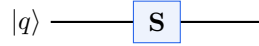
$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle.$$

```
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
```

Input	Measurement distribution	State before measurement
0	50% 0, 50% 1	$(0\rangle + 1\rangle)/\sqrt{2}$
1	50% 0, 50% 1	$(0\rangle - 1\rangle)/\sqrt{2}$

Equal classical probabilities hide the relative minus sign responsible for interference.

4.5 S phase (S)

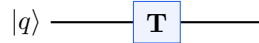


$$S = P(\pi/2) = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \quad S|1\rangle = i|1\rangle, \quad S^2 = Z.$$

```
S -I Q -0 Q
```

Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$i 1\rangle$

4.6 T phase (T)

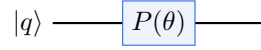


$$T = P(\pi/4) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \quad T^2 = S, \quad T^4 = Z.$$

```
T -I Q -0 Q
```

Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$e^{i\pi/4} 1\rangle$

4.7 Arbitrary phase (PHASE)



$$P(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}, \quad \alpha|0\rangle + \beta|1\rangle \mapsto \alpha|0\rangle + \beta e^{i\theta}|1\rangle.$$

The angle is embedded in the opcode:

```
PHASE=1.5708 -I Q -0 Q      # radians
PHASE=3/2 -I Q -0 Q        # rational radians
PHASE=90d -I Q -0 Q        # degrees
PHASE=45/2d -I Q -0 Q      # rational degrees
PHASE=pi -I Q -0 Q
PHASE=-11pi/6 -I Q -0 Q
PHASE=2*pi/3 -I Q -0 Q
BPHASE=pi/4 -I Q -0 Q      # P(-pi/4)
```

Accepted numeric atoms include signed integers, decimals, leading-dot decimals, and one optional rational denominator. Pi notation accepts an optional sign, an integer coefficient, an optional `*`, and an optional positive integer denominator. A zero denominator and malformed angle raise an error.

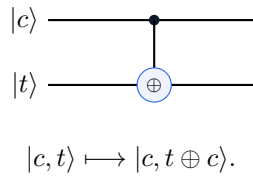
Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$e^{i\theta} 1\rangle$

4.8 Backward-prefix syntax

The AST recognizes a leading B before any registered primitive gate name, for example BX, BH, BCNOT, or BSWAP, and sets `reverse=true`. Only `BPHASE=angle` currently changes emitted behavior: the compiler negates the phase angle. For every other gate the flag is presently metadata only. Many gates are self-inverse (X , H , CNOT, CCNOT, CZ, SWAP), but S, T, Y, and CY inverses are *not* synthesized from the reverse flag. Do not rely on those B-prefixed forms for inverse execution.

5 Controlled and two-qubit primitive gates

5.1 Controlled NOT (CNOT)

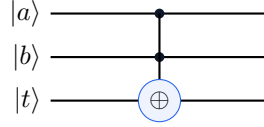


```
CNOT -I Control -0 Target
```

c	t	c'	t'
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

CNOT is a reversible embedding of XOR because it preserves the control.

5.2 Doubly controlled NOT / Toffoli (CCNOT)



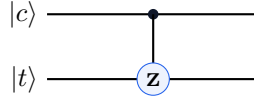
$$|a, b, t\rangle \mapsto |a, b, t \oplus (a \wedge b)\rangle.$$

CCNOT -I A B -O Target

a	b	t	a'	b'	t'
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

With a target initialized to zero, Toffoli computes classical AND reversibly.

5.3 Controlled Z (CZ)



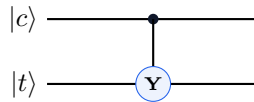
$$\text{CZ}|c, t\rangle = (-1)^{ct}|c, t\rangle, \quad \text{CZ} = \text{diag}(1, 1, 1, -1).$$

CZ -I Control -O Target

Input	Measured output	Amplitude multiplier	State
00	00	+1	$ 00\rangle$
01	01	+1	$ 01\rangle$
10	10	+1	$ 10\rangle$
11	11	-1	$- 11\rangle$

The classical columns are unchanged; the quantum phase is the operation.

5.4 Controlled Y (CY)

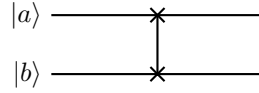


$$|0, t\rangle \mapsto |0, t\rangle, \quad |1, 0\rangle \mapsto i|1, 1\rangle, \quad |1, 1\rangle \mapsto -i|1, 0\rangle.$$

CY -I Control -0 Target

c	t	c'	t'	Quantum phase
0	0	0	0	+1
0	1	0	1	+1
1	0	1	1	i
1	1	1	0	$-i$

5.5 Swap (SWAP)



$$\text{SWAP}|a, b\rangle = |b, a\rangle.$$

SWAP -I A B -0 A B

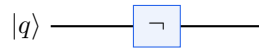
The two output tokens are mandatory and must resolve to the same two wires, in the same order, as the inputs.

a	b	a'	b'
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

6 Derived logical gates

Derived gates are direct AST operations implemented as reversible target mutations. Inputs are preserved. If target t starts in zero, AND/OR/XOR write the familiar classical function. If it does not, the true behavior is the XOR embedding shown below.

6.1 Logical NOT (NOT)



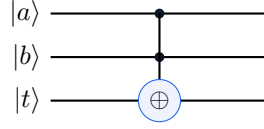
$$t' = t \oplus 1.$$

NOT -I Target -0 Target

t	t'
0	1
1	0

The current gate implementation mutates the output target. Use the same token for input and output; a distinct input token is not copied.

6.2 Reversible AND (AND)

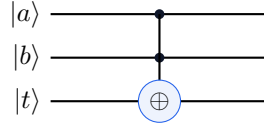


$$t' = t \oplus (a \wedge b).$$

SET Out Op
AND -I A B -O Out

a	b	$t = 0$	$t' = a \wedge b$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	1

6.3 Reversible NAND (NAND)

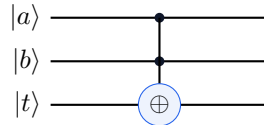


$$t' = t \oplus (a \wedge b) \oplus 1.$$

SET Out Op
NAND -I A B -O Out

a	b	$t = 0$	$t' = \neg(a \wedge b)$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	0	0

6.4 Reversible OR (OR)



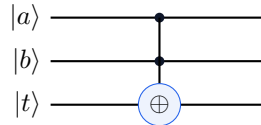
$$t' = t \oplus (a \vee b).$$

SET Out Op
OR -I A B -O Out

a	b	$t = 0$	$t' = a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	1

For bits, $a \vee b = a \oplus b \oplus (a \wedge b)$.

6.5 Reversible XOR (XOR)



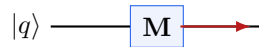
$$t' = t \oplus a \oplus b.$$

```
SET Out Op
XOR -I A B -O Out
```

a	b	$t = 0$	$t' = a \oplus b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	0

7 Measurement gate

7.1 Measurement (MEASURE)



```
MEASURE -I Q      # exactly one named wire
MEASURE           # every wire known at this point
```

MEASURE takes no -O. Repeated one-wire commands are useful when only declared returns should become classical. A bare command emits one measurement operation per currently allocated wire.

State	Outcome probabilities	Collapsed state
$ 0\rangle$	$P(0) = 1$	$ 0\rangle$
$ 1\rangle$	$P(1) = 1$	$ 1\rangle$
$\alpha 0\rangle + \beta 1\rangle$	$P(0) = \alpha ^2, P(1) = \beta ^2$	$ 0\rangle$ or $ 1\rangle$

8 State, token, and cycle commands

8.1 SET — initialize or alias (lowered/executed)

```
SET Q 0p
SET Q 1p
SET Q sp
SET Local $Parameter
```

SET target value requires a positional value. A constant schedules or emits initialization: zero, X from zero, or H from zero. A nonconstant value makes the target an alias of that source. A state parameter's constant SET is treated as a runtime default, allowing the UI or truth-table runner to supply its actual start state.

8.2 CREATETOKEN — reserve wires (lowered/executed)

```
CREATETOKEN -I Sum Carry Workspace
```

Every -I token immediately claims a stable wire index. No -0 is used. This is useful for declared outputs before their first gate.

8.3 DELETETOKEN and FREE — lifetime hints (parsed only)

```
DELETETOKEN -I Workspace
FREE -I ScratchA ScratchB
```

Both commands are accepted and logged. They do not currently remove, reset, or reuse a wire; later compaction removes symbolic registers that never participate in an emitted gate or exposed parameter.

8.4 INCREASECYCLE — cycle boundary (lowered/executed)

```
INCREASECYCLE
```

The compiler flushes pending zero-initializations into an internal batched RESET, increments the cycle counter, and prepares workspace for following operations. Cycle suffixes on tokens do not create separate visual wires.

8.5 JOIN and SPLIT (parsed only)

```
JOIN -I A B -O AB
SPLIT AB A 2
```

The intended mathematical model for a clean join is $|AB\rangle = |A\rangle \otimes |B\rangle$. A split into independent pure states is only possible when the composite state factors. The current compiler merely logs these rows: no tensor-product allocation, extraction, gate, or alias is emitted. Because these are non-gate commands, the parser does not enforce their arity.

9 Process composition and return values

9.1 MAIN-PROCESS (lowered/executed)

```
MAIN-PROCESS FullAdder
```

This marks the process body and supplies its catalog/library name. Compilation already begins from the parsed process, so the row itself emits no gate.

9.2 DECLARECHILD (parsed only)

```
DECLARECHILD SingleBitFullAdder
```

The declaration is logged. It does not load inline source. Child bodies must exist in the external `librarySources` catalog when called.

9.3 RUNCHILD and CALL (lowered/executed)

```
RUNCHILD SingleBitFullAdder \
  -I A B Cin \
  -O Carry Sum

CALL SingleBitFullAdder -I A B Cin -O Carry Sum
```

The two commands currently have the same expansion behavior. The first positional argument is the child process name. Inputs bind by PARAMS order; outputs bind by the child's RETURNVALS order. Before expansion, each bound parent output is zero-prepared once. Child-local names receive a unique scope.

9.4 RETURNVALS (lowered/executed)

```
RETURNVALS Carry Sum
```

Positional tokens define the process's ordered public outputs. They determine child output binding, truth-table output columns, and the UI's preferred logical width. Do not use `-I` or `-O` here: flags would be retained as positional return tokens.

9.5 ACCEPTVALS (lowered/executed)

```
CALL Child -I A B
ACCEPTVALS LocalCarry LocalSum
```

Each positional local token aliases the corresponding value from the most recent child return list. Direct `-O` binding on RUNCHILD/CALL is clearer when output names are known.

9.6 Nested full-adder example

```
# child.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS ChildAdder
CREATETOKEN -I Carry Sum
SET Carry Op
SET Sum Op
XOR  -I A B    -O Sum
CNOT -I Cin    -O Sum
CCNOT -I A B    -O Carry
CCNOT -I A Cin -O Carry
CCNOT -I B Cin -O Carry
RETURNVALS Carry Sum
```

```
# parent.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS Parent
CREATETOKEN -I Cout Result
RUNCHILD ChildAdder -I A B Cin -O Cout Result
MEASURE -I Cout
MEASURE -I Result
RETURNVALS Cout Result
```

For basis inputs, the full-adder equations are

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

<i>A</i>	<i>B</i>	<i>C_{in}</i>	<i>C_{out}</i>	<i>Sum</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

10 Accepted compatibility commands

The following rows are valid AST commands but currently produce log entries only. Their arguments are retained as whitespace-separated strings and their arity is not validated.

Command	Example	Current effect
COMPILEPROCESS	COMPILEPROCESS Child	Logs process-control text; does not load or compile a child. Use RUN-CHILD/CALL with library source.
MASTerval	MASTerval Result	Logs a final/master-value marker; does not alter RETURNVALS.
SAVE_STATE	SAVE_STATE checkpointA	Logs a checkpoint request; no state snapshot is created.
LOAD_STATE	LOAD_STATE checkpointA	Logs a restore request; no state is changed.

11 Truth-table companion syntax

Truth tables are metadata paired to a process; they are not .qpucir AST commands. A text .qpuio file may use spaces or commas:

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

- MAIN-PROCESS: `name` and a column row beginning with `#` are required.
- Explicit INPUTS: and OUTPUTS: remove ambiguity. Without them, the parser can use matching PARAMS/RETURNVALS from the protocol.
- Data rows begin at index 0 and must be sequential.
- Every row must match the header width and every cell must be `0p`, `1p`, or `sp`.
- Comma-separated headers and rows are also accepted.
- Partial tables may list fewer than 2^n distinct input patterns, but not zero rows, duplicates, or more than 2^n rows.

The JSON envelope form is:

```
{
  "format": "qpuio",
  "version": 1,
  "processName": "HalfAdder",
  "inputColumns": ["A", "B"],
  "outputColumns": ["Carry", "Sum"],
  "rows": [
    ["0p", "0p", "0p", "0p"],
    ["0p", "1p", "0p", "1p"],
    ["1p", "0p", "0p", "1p"],
    ["1p", "1p", "1p", "0p"]
  ]
}
```

The table runner initializes PARAMS by column order, executes the circuit, measures RETURNVALS, and compares those bits only. An expected output **sp** accepts either measured bit; an input **sp** initializes an equal superposition before execution.

12 Complete worked protocol

```
PARAMS: A:state B:state Theta:float

MAIN-PROCESS BellAndLogic
CREATETOKEN -I Q0 Q1 AndOut

SET Q0 $A
SET Q1 $B
SET AndOut Op

# Quantum segment
H -I Q0 -O Q0
CNOT -I Q0 -O Q1
PHASE=pi/4 -I Q1 -O Q1

# Basis-state comparison output
AND -I A B -O AndOut

MEASURE -I Q0
MEASURE -I Q1
MEASURE -I AndOut
RETURNVALS Q0 Q1 AndOut
```

Starting from $|00\rangle$, the H-CNOT segment prepares

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

The phase gate then produces

$$\frac{|00\rangle + e^{i\pi/4}|11\rangle}{\sqrt{2}}.$$

The two quantum measurements are correlated, while **AndOut** compares the basis inputs through $A \wedge B$. A deterministic truth table can specify the logic output, but repeated simulation is needed to characterize Bell-state measurement probabilities.

13 Compiler output and lowering details

Compilation returns:

```
CompileResult {
  gates: CircuitGate[]
  qubitCount: number
  logicalQubitCount: number
  parsed: ParsedCommand[]
  log: string[]
  tokenMap: Record<string, number>
  processParams: ProcessParam[]
  returnValues: ReturnValue[]
}
```

Each parsed command records `op`, original `raw` text, `inputs`, `outputs`, positional `args`, optional normalized radian `phase`, `reverse`, and `noParameterSubstitution`. Each emitted gate records `id`, `type`, ordered `step`, `targets`, `controls`, optional `phase`, and source text.

After child expansion, unused symbolic holes are compacted to dense qubit indices. Internal RESET gates count for simulation but are filtered from the visible circuit. UI logical width prefers RETURNVALS count, then exposed PARAMS count, then physical simulator width.

14 Error conditions and practical rules

1. Unknown opcodes fail with `Unknown command`. RESET is not a source opcode.
2. Every AST gate except MEASURE requires `-I` and `-O`, then exact registry arity is checked.
3. PHASE requires a valid nonzero-denominator angle after `=`.
4. SWAP requires two matching input/output wires.
5. RUNCHILD/CALL fails if the named process is absent from the library.
6. SET fails when its positional value is missing.
7. RETURNVALS order is semantically significant for child bindings and truth-table columns.
8. Use the same input/output token for single-qubit gates.
9. Initialize derived-gate targets deliberately; their operation is XOR into the existing target, not destructive assignment.
10. Treat parsed-only commands as compatibility annotations until their runtime lowering is implemented.

15 Quick-reference grammar

This is a practical description of the accepted surface language rather than a separate parser generator grammar:

```
protocol      := [params-line] logical-line*
params-line   := "PARAMS:" (name ":" type)*
logical-line  := command | comment | blank

command       := opcode argument*
opcode        := supported-command
               | primitive-gate
               | "B" primitive-gate
               | "PHASE=" angle
               | "BPHASE=" angle

gate-row      := opcode "-I" input+ "-O" output+ ["-$R"]
measure-row   := "MEASURE" ["-I" input]
set-row       := "SET" target value
process-row   := "MAIN-PROCESS" name
child-row     := ("RUNCHILD" | "CALL") name
               ["-I" argument*] ["-O" argument*]
returns-row   := "RETURNVALS" token*
accept-row    := "ACCEPTVALS" token*

token         := ["$"] name [":" suffix]
```

```

constant      := ("0p" | "1p" | "sp") ["_dim" integer]
angle         := rational
              | rational "d"
              | [sign] [integer ["*"]] "pi" ["/" integer]

```

Parser detail: PHASE= pi/2 is joined into one opcode/value pair, but PHASE = pi/2 is not. Operations and flags are case-insensitive; names, types, and positional values are otherwise retained.

16 Operation checklist

Item	Covered in	Item	Covered in
PARAMS:	Lexical syntax	MAIN-PROCESS	Process composition
X,Y,Z,H	Single-qubit gates	S,T,PHASE	Single-qubit gates
CNOT,CCNOT	Controlled gates	CZ,CY,SWAP	Controlled gates
NOT,AND,NAND	Derived gates	OR,XOR	Derived gates
MEASURE	Measurement	SET	State/token commands
CREATETOKEN	State/token commands	DELETETOKEN,FREE	State/token commands
INCREASECYCLE	State/token commands	JOIN,SPLIT	State/token commands
DECLARECHILD	Process composition	RUNCHILD,CALL	Process composition
RETURNVALS	Process composition	ACCEPTVALS	Process composition
COMPILEPROCESS	Compatibility	MASTerval	Compatibility
SAVE_STATE	Compatibility	LOAD_STATE	Compatibility
-I,-O,-\$R	Flags	B prefix	Single-qubit/reverse syntax
comments,\	Lexical syntax	.qpuio	Truth-table syntax